

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/348335709>

Visual-based Autonomous Unmanned Aerial Vehicle for Inspection in Indoor Environments

Conference Paper · November 2020

DOI: 10.1109/LARS/SBR/WRE51543.2020.9307024

CITATIONS

20

READS

457

11 authors, including:



Ricardo Grando

Technological University of Uruguay

72 PUBLICATIONS 423 CITATIONS

SEE PROFILE



Pedro Miranda Pinheiro

Federal University of Rio Grande

22 PUBLICATIONS 270 CITATIONS

SEE PROFILE



Cesar Silva

State University of Campinas (UNICAMP)

10 PUBLICATIONS 165 CITATIONS

SEE PROFILE



Vivian Misaki Aoki

Federal University of Rio Grande

9 PUBLICATIONS 85 CITATIONS

SEE PROFILE

Visual-based Autonomous Unmanned Aerial Vehicle for Inspection in Indoor Environments

Ricardo B. Grando¹, Pedro M. Pinheiro¹, Nicolas P. Bortoluzzi¹, César B. da Silva², Otávio F. Zauk¹, Mariana O. Piñeiro¹, Vivian M. Aoki¹, André L. S. Kelbouscas¹, Ylanna B. Lima¹, Paulo L. J. Drews-Jr¹, Armando A. Neto³

Abstract—This paper presents a visual-based autonomous Unmanned Aerial Vehicle (UAV) capable to perform a broad range of navigation tasks in indoor environments. Unlike other related vehicles, our robot does not need an external operator whatsoever or any kind of off-board processing, reducing the overall workload and manual dependency. We evaluated our vehicle in a real indoor environment that simulates an offshore oil platform scenario, demonstrating that it is capable of localize itself, maintaining stable flight, navigating, detecting landing spots, precisely landing on them, and detecting and reading information from the scenario. The methodology presented in this work was used to win the 2019 Brazil Open Flying Robot League during the 2019 Latin America Robotics Competition, among eleven teams and outscoring the second-placing team by almost a factor of two.

Index Terms—Autonomous Unmanned Aerial Vehicles, Visual Inspection, Indoor Navigation, Visual Odometry.

I. INTRODUCTION

In the last few years, the use of Unmanned Aerial Vehicles (UAVs) has become increasingly popular in tasks of some fields like industry, research and military [1] [2]. Advances in technologies related to 3D printing, sensing, embedded processing and battery efficiency and miniaturization allowed a broader development of smaller and more efficient aerial vehicles [3]. The majority of commercial UAVs are designed to operate in outdoor environments with its software system heavily fitted and dependent on accurate GPS data feedback, either to find and maintain the vehicle's correct current position or to navigate to a new desired waypoint [1] [2]. However, the amount of UAVs with the ability to navigate in indoor environments is quite small, mainly because of the high dependence on GPS which is either unreliable or unavailable in scenarios of this kind [4].

The inspection of hazardous and damaged buildings, wind turbines, urban canyons and oil platforms are some of the scenarios where GPS signal is constrained and operating autonomous UAV can be useful [5, 6]. UAVs in these tasks

are often manually employed by an external human operator which relies on most cases solely on a video-system based transmission to control the vehicle and navigate through the flight's objectives [2]. Besides the fact of being difficult for a human operator to achieve a necessary awareness to perform such tasks, a failure of communication with the vehicle to receive image data and send command can result in loss of data, inaccurate navigation and even crashes [7]. Therefore, these issues are important to be addressed by researchers on the development of UAVs nowadays [8].

This work aims to overcome these issues by developing an autonomous visual-based UAV with embedded online intelligence capable to maintain stable flight, navigate and locate itself in indoor environments. Our approach does not need an external operator and any kind of off-board processing, reducing the overall workload and manual dependency that some tasks may demand. We evaluated our vehicle in a real indoor environment that simulates oil platforms. The methodology presented in this work was used to win the 2019 Brazil Open Flying Robot League during the 2019 Latin America Robotics Competition, outscoring the second-placing team by almost a factor of two. Figure 1 shows our vehicle performing during the competition.



Fig. 1: Vehicle performing in the 2019 Brazil Open Flying Robot League.

The main contributions of this work are listed as follows:

- We present a visual-based autonomous UAV for real indoor environments with embedded online intelligence capable to maintain stable flight, navigate, locate itself and perform tasks like autonomous landing and text and color-based sensing detection.

¹P. L. J. Drews-Jr, Ricardo B. Grando, Pedro M. Pinheiro, Nicolas P. Bortoluzzi, Otávio Zauk, Mariana O. Pineiro, Vivian M. Aoki and Andre L. S. Kelbouscas are with the NAUTEC, Centro de Ciências Computacionais, Universidade Federal do Rio Grande - FURG, RS, Brazil. E-mail: paulodrews@furg.br

²César B. da Silva is with Universidade de Campinas, SP, Brazil. E-mail: cesar.silva2612@gmail.com

³A. Alves Neto is with the Electronic Engineering Department, Universidade Federal de Minas Gerais, MG, Brazil. E-mail: aaneto@cpdee.ufmg.br

- We provide the code ¹ of our vehicle's system, including our ROS-based mission planner and text and color-based sensing packages.
- We also provide a simulation of our vehicle described on the Gazebo simulator adopted to tune the system of our real vehicle.

The work is organized as follows: Sec. II discusses related works. Sec. III presents our approach. The obtained results are presented in Sec. IV,. Finally, Sec. V summarizes our contributions and outlines our future works.

II. RELATED WORK

Schäfer *et al.* [2] proposed an UAV for indoor inspection of a wind turbine. It performed the inspection by using a priori 3D mapping of the plant with a system based on the ROS framework [9]. They focused on collision avoidance by using the information of 2D LIDAR instead of a vision-based system.

Winkvist *et al.* [1] designed a semi-autonomous aerial vehicle for internal inspection of large structures. It focused on determining the vehicle's correct position and velocity by using SLAM. They also used a ground station to track down the vehicle and a human instructor to control the vehicle.

Indoor visual inspection was also explored by the vehicle proposed by Ortiz *et al.* [10]. The work focused on developing a small UAV capable to inspect vessels by making the use of SLAM and the ROS framework. They also developed a ROS-based mission planner to perform visual detection tasks such as corrosion detection and coating breakdown.

Vessel maintenance was also addressed in the work of Bonnin *et al.* [11] by developing a semi-autonomous UAV. The vehicle provided images of real-time detection to a base station for later processing. Their overall system design and the tests performed are similar to Ortiz *et al.* [10, 12], the last proposed later on.

Our proposed vehicle was also designed to perform indoor visual inspection, to rely on visual information to locate itself and to autonomously navigate. However, our approach focus on an autonomous UAV that does not make the use of an external operator or off-board processing. Our approach differs from the previously discussed works due to the use of visual odometry for localization. We also evaluated our vehicle is an indoor environment that simulates commonly tasks on an oil platform.

III. METHODOLOGY

In this Section, we discuss how our vehicle was developed and the tool we made use of. We detail the overall mechanical and hardware setup with its sensing coupled as well as the vision-based system for autonomous navigation and detection.

A. Robot Operating System

The Robot Operating System (ROS) [9] is a framework for developing applications for robots. It is a collection of

tools and libraries that provides operating system standard services, such as hardware abstraction, device low-level control and package management. The flow of ROS processes is represented by graphs where each application is a node of the system. The nodes receive and send messages to perform sensing, control, planning and other tasks common to a robot's software system demands. ROS is not a real-time operating system, although a real-time application can be used by third-party tools. Low latency is important for robots and the lacking of real-time support is being addressed on the development of the second version of ROS. For this work, it was used the Melodic version of ROS alongside some supported libraries such as Mavros [13] and OpenCV [14].

B. Ardupilot and PixHawk board

The PixHawk board [15] is a hardware design that can be used for autonomous flight to perform navigation-related tasks such as surveillance and data acquisition. Its toolkit is based on a lightweight protocol called MAVLink, which scales from serial to UDP links. It can serve also as a communication protocol between an onboard computer and a ground station.

Besides providing the MAVLink middleware layer, its multiple layered architecture also has a communication layer with ROS implemented, turning it into a convenient high-level programming interface. The hardware supports a range of sensing including radio telemetry and GPS.

Ardupilot [16] provides firmware for the PixHawk board. Since it is open-source, it is also possible to edit and compile the firmware with the developer's own specifics needs. The source code provides a development and simulation platform for all its four supported vehicle categories, including Arducopter for aerial vehicles.

C. Design Overview

Our vehicle was designed in such a way that it allowed all processing to be done embedded in it, avoiding the use of external processing on a ground station, which is common in commercial UAVs. It was done so to prevent any loss of signal to the vehicle due to bad communication or interference of any kind, which could make the vehicle unstable leading to a crash. The diameter of the vehicle was set to 45cm with a 3D printed center frame for coupling a quadrotor-based existing frame, as shown in Figure 2.

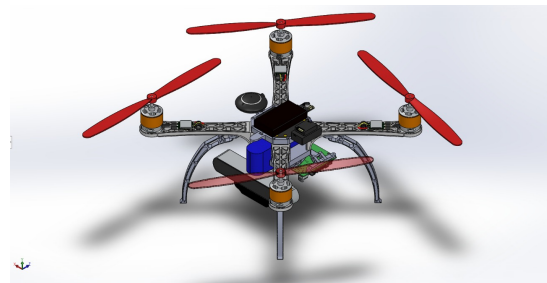


Fig. 2: Prototype of our vehicle in a computer aided design (CAD) assembly.

¹<https://github.com/Nicolasbort/hydrone-LARS>

The Pixhawk board was mounted in the upper part of the center frame in such a way that it was parallel to the vehicle, making the standard orientation of the Pixhawk the same as the vehicle. The propellers are 10 inches in diameter and each rotor had a total capacity of 800g of lift, a total of 3.2kg for the vehicle.

We also embedded the computation hardware necessary for indoor localization. It was chosen a NVIDIA Jetson Nano for embedding processing, since it supports a high rate of image processing demanded by the software due to its CUDA support and has a very low power consumption with around 10W. The Jetson Nano is settled in the downward-back part of the vehicle taking into the account the center of mass of the vehicle, once a ZED camera was settled in the downward-front of the vehicle. Another camera, a Logitech C920 is also fixed in the down part of the vehicle, rotated 90 degrees in the Y-axis in such a way that it pointed downwards to the floor. The image with the vehicle mounted can be seen in Figure 1 and the sensors used can be analyzed in Table I.

D. Vehicle System

For the software development, the first thing was to find a way to replace the use of GPS. That was done by changing the Ardupilot's source code and compiling a firmware version that uses feedback from pose yielding algorithm instead of GPS. After evaluated several state-of-art techniques of SLAM, visual odometry and visual-inertial odometry, we decided to use a combination of visual odometry and a stereo camera to get the robot's pose. The camera chosen was the ZED 2K Stereo camera wrapper [17] because of its superior performance [18]. It was also used taking into account the fact that the wrapper is implemented to run with CUDA and is fully compatible with the Jetson Nano. In the overall architecture, it was used the Mavros package to communicate the Pixhawk with the Jetson Nano and the ZED wrapper was used to communicate the ZED camera with the NANO. Figure 3 shows the final architecture for the system.

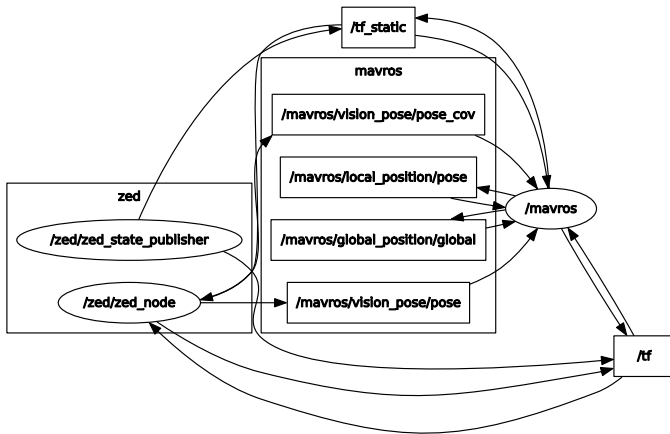


Fig. 3: ROS-based system architecture.

The following step was the development of a package to make an interface with Mavros to send commands to

TABLE I: Vehicle's description with sensors and details about them.

Device	Details
Custom Frame	A standard F450 frame was used with a custom central part, which allowed for a larger battery pack being placed close to the plane of the arms of the vehicle, and also allowed for custom camera supports being mounted on it. The frame of the vehicle was designed to fit inside a square of 45cm of diagonal. Mass: 395g
Propellers	Four 10inx4.5in plastic propellers were used. Mass: 48g
Motors	Four 1000kv brushless motors were used. Each motor and propeller combination was capable of generating 800g of thrust, thus generating a total of 3200g of lift. Mass: 212g
Electronic Speed Controllers	Four 30A ESCs were used. Mass: 128g
Battery	A 6000mAh 4S2P lithium polymer battery pack was used and provided a flight time of roughly 8min, depending on the performed tasks. Mass: 476g
Flight Controller	A Pixhawk 1 is used for vehicle control. It has an internal IMU and barometer, and low-power 180MHz microcontroller. The flight controller sensors provide pose estimation that is fused with the visual odometry output. Mass: 38g
Monocular Camera	A Logitech C920 camera was used for visual recognition. Its support was replaced for a custom 3D printed one for a lower mass Mass: 100g
Stereo Camera	A ZED Stereo Camera is used for visual odometry. It is capable of providing 2x1080p video output, 100Hz depth sensing from 0.3m up to 25m and 100Hz pose estimation. Mass: 135g
Computer	A Jetson Nano Developer Kit is used for pose calculation, and communication with the flight controller. Mass: 141g

the vehicle, allowing it to have an intelligence and work autonomously. A *mission_planner_package*² was developed to send commands and get the state of the vehicle. Complementary packages to perform the inspection in the environment were also developed to run on the Jetson Nano board alongside the *mission_planner_package*.

E. Vehicle Simulation

Robotics is an expensive field of study that requires many resources that usually are difficult to fully obtain. Robot simulation is an essential and common way to develop many applications for robots and makes it possible to design robots and test applications. Gazebo [19] makes it possible by having an active community and being fully compatible with ROS.

²https://github.com/ricardoGrando/hydrone_mission_planner

Many frameworks were developed to support the use of Gazebo for several types of vehicles and environments. Among them, the RotorS simulator [20] is a simulation framework that allows researching on aerial vehicles using the Gazebo environment. We simulated our vehicle and described the real environment to test our approach based on RotorS. The vehicle described can be seen in Figure 4.

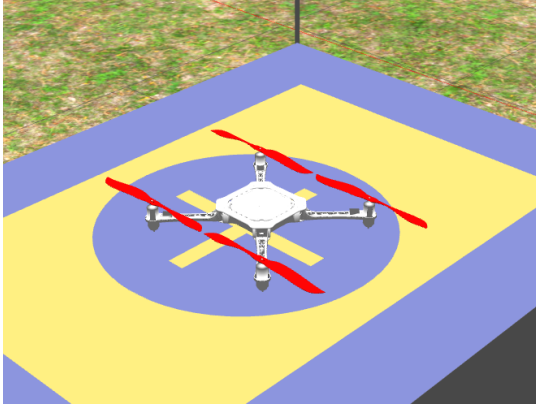


Fig. 4: Vehicle performing in the simulated scenario.

IV. RESULTS

We made use of the 2019 Brazil Open Flying Robot League competition environment to evaluate our vehicle and system, which had a simulated oil platform scenario. Fig. 5 shows the scenario dimensions (5a), the competition scenario (5b) and the described scenario on Gazebo simulator (5c).

The scenario consists of a start base (corner left-down Fig. 5a), two hanging landing bases of $1m$ wide and $1.5m$ elevated from the ground, three landing bases of $1m$ wide placed on the ground, and an orange pipe of $7m$ high and with $0.2m$ of radius placed on the ground as well. The scenario was set with 4×4 meters wide and $8m$ in height. The overall performance of our vehicle in simulation can also be seen in a complementary video³.

The first task to evaluate our vehicles is the navigation. In this task, the vehicle should autonomously go from the start base and land at one of the hanging bases. We meant to analyze the vehicle's vision-based localization and navigation capability, as well as the ability to correctly detect a base and adjust itself to perform the landing. This task aims to simulate a basic task for an autonomous UAV in an oil platform or any other environment, which is to locate itself, navigate and land safely at a desired location. We converted the image color space to detect the base and reduce the effect of the light, isolated the yellow and blue colors in two new images, filled it and made a *bitwise* process that removes every pixel that is not in both. After that, we looked for a remaining rectangle and published the distance, in pixels, of it to the vehicle in a ROS topic.

First we set two waypoints: $(0.0, 0.0, 2.0)$ and $(2.75, 0.0, 2.0)$. The first waypoint was set to be $1.5m$ up from the start base and the second on the top of the hanging base's center. When reaching the second waypoint, we set the vehicle to use the base detection to adjust its position and land properly on the base. Fig. 6 shows an image sequence of real vehicle performing the task on the competition scenario, while the Fig. 7 show the simulated vehicle performing the task. The path performed by the vehicle and the expected path can be seen in Fig. 8, while the base detection can be seen in Fig. 9.

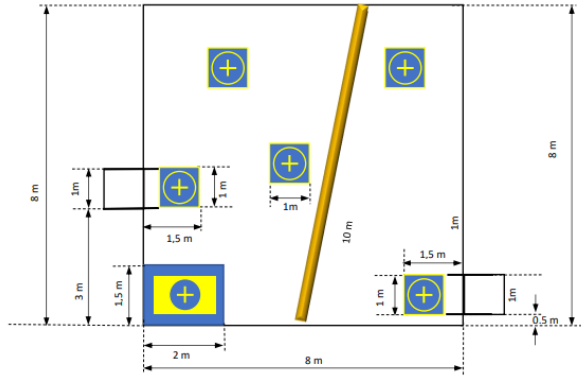
Besides locating itself, navigating and landing safely at a desired location, the UAV must be able to be capable to perform some indoor inspection in an oil platform or any other location. The capability is related to detect the state of a sensor by autonomously reading some information. Therefore, we developed an image-based sensing detection system to detect the state of a sensor.

In the second task, the vehicle should go through the pipe can and find damaged sensors. They were highlighted with a red color. The vehicle should leave the start base and go to one of the corners of the pipe can and go through it. We also developed a led-based system to make the external visualization in real-time in case a damaged sensor is detected. So, everybody looking at the vehicle could see it by the blinking led if a bad sensor was detected. Fig. 10c shows the vehicle signaling a bad detection and the Figures 10b and 10a show the detection on the real and simulated scenarios, respectively.

A final task was proposed to test our vehicle. In this task, the vehicle should detect the information displayed by a simulated gas sensor. This information was set with a numeric format and placed over the same hanging base used for the first task. So, the vehicle should go over the hanging base and read the sensor using its camera pointed to the ground. The vehicle should either blink the green led when the values were within a range, defined by the competition, or the red led when the values were out of the range. We evaluated two types of algorithms for number detection: Tesseract OCR [21], which is excellent but it is computationally expensive, and K-Nearest Neighbors (KNN) [22] that is a great classification method and perform better together with other essentials algorithms, like SLAM. Thus, we choose the KNN algorithm to complete this task.

We converted the image color space to isolate the sensor, cropped the base, maintained just black color pixels and cropped the sensor pixel. In order to check if the sensor was in the correct orientation, we based our detection on looking at each frame for the percent symbol and rotating the frame if the symbol was not in the correct position. We created a dataset of images of 7-segment numbers and percent symbols to generate data to train the KNN. The detected numbers are saved into a log and further collected. Fig. 11 shows vehicle performing the reading.

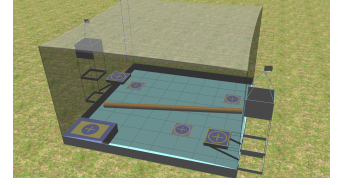
³https://www.youtube.com/watch?v=KcecohS_rHA



(a) Environment specs.



(b) Real environment (Competition).



(c) Simulated environment.

Fig. 5: Testbed scenario: specifications and illustration.

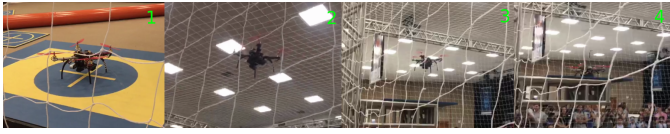


Fig. 6: Sequence of the navigation task performed by the real vehicle.

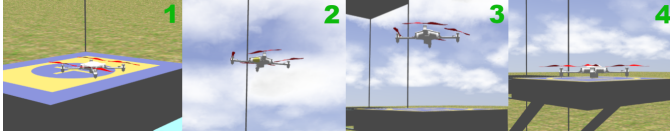


Fig. 7: Sequence of the navigation task performed by the simulated vehicle.

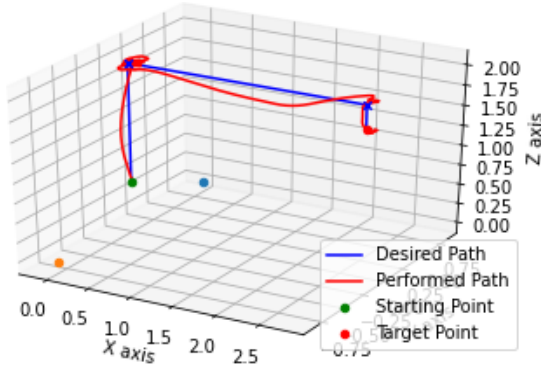


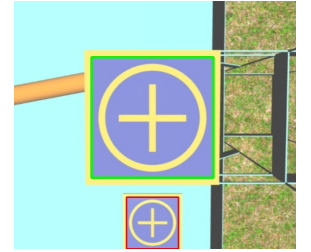
Fig. 8: Path performed by the simulated vehicle in the navigation task.

V. CONCLUSIONS

In this work, we presented an autonomous visual-based UAV with embedded online intelligence capable of performing tasks in indoor environments. Unlike other related vehicles, we designed our prototype in such a way that there is no need for an external operator and any kind of off-board



(a) Real scenario.



(b) Described vehicle.

Fig. 9: Base detection performed by the vehicle.



(a) Detection simulated scenario.



(b) Detection real scenario.

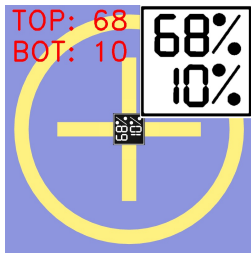


(c) External view of the detection. Red blinking led showing the bad sensor detection.

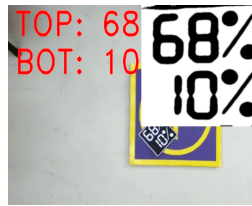
Fig. 10: Bad sensors detection task.

processing at any point. We focused on reducing the overall workload and manual dependency that tasks in this kind of environment may demand. We evaluated our real vehicle in an indoor environment that simulates an oil platform scenario and described it on the Gazebo simulator to facilitate the development of the software system.

The developed system is capable of localize itself, maintain stable flight, navigate, detect landing points, and precisely



(a) Detection simulated scenario



(b) Detection real scenario

Fig. 11: Sensor reading task performed by the vehicle.

land on it and detect and read information from the scenario. All the software system was developed based on open-source frameworks, such as ROS and Ardupilot, and are entirely provided by the authors. Our strategy was employed to win the 2019 Brazil Open Flying Robot League during the 2019 Latin American Robotics Competition, outscoring the second-placing team by almost a factor of two.

Future works will be focused in the improvement of the vehicle embedded system to increase its overall processing capability and autonomy. We also aim to evaluate our vehicle in several indoor environments with other tasks. New path planning and control strategies can also be applied to fulfill other missions requirement.

ACKNOWLEDGMENT

The authors would like to thank to National Council for Scientific and Technological Development (CNPq), the Coordination for the Improvement of Higher Education Personnel (CAPES) - Finance Code 001, FAPERGS, FAPEMIG, RoboCup Brazil and PRH-ANP.

REFERENCES

- [1] S. Winkvist, E. Rushforth, and K. Young, "Towards an autonomous indoor aerial inspection vehicle," *Industrial Robot: An International Journal*, 2013.
- [2] B. E. Schäfer, D. Picchi, T. Engelhardt, and D. Abel, "Multicopter unmanned aerial vehicle for automated inspection of wind turbines," in *Mediterranean Conference on Control and Automation (MED)*, 2016, pp. 244–249.
- [3] A. C. Horn, P. M. Pinheiro, C. B. Silva, A. A. Neto, and P. L. J. Drews-Jr, "A study on configuration of propellers for multirotor-like hybrid aerial-aquatic vehicles," in *ICAR*, 2019, pp. 173–178.
- [4] A. W. Vieira, P. L. J. Drews-Jr, and M. F. M. Campos, "Spatial density patterns for efficient change detection in 3d environment for autonomous surveillance robots," *IEEE T-ASE*, vol. 11, no. 3, pp. 766–774, 2014.
- [5] P. L. J. Drews-Jr, A. A. Neto, and M. F. M. Campos, "Hybrid unmanned aerial underwater vehicle: Modeling and simulation," in *IEEE/RSJ IROS*, 2014, pp. 4637–4642.
- [6] A. A. Neto, L. A. Mozelli, P. L. J. Drews-Jr, and M. F. M. Campos, "Attitude control for an hybrid unmanned aerial underwater vehicle: A robust switched strategy with global stability," in *IEEE ICRA*, 2015, pp. 395–400.
- [7] P. L. J. Drews-Jr, A. A. Neto, and M. F. M. Campos, "A survey on aerial submersible vehicles," in *Conference: IEEE/OES Oceans*, 2009, pp. 4244–2523.
- [8] R. T. S. da Rosa, P. J. D. O. Evald, P. L. J. Drews-Jr, A. A. Neto, A. C. Horn, R. Z. Azzolin, and S. S. C. Botelho, "A comparative study on sigma-point kalman filters for trajectory estimation of hybrid aerial-aquatic vehicles," in *IEEE/RSJ IROS*, 2018, pp. 7460–7465.
- [9] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: an open-source robot operating system," in *IEEE ICRA - Workshop on open source software*, vol. 3, no. 3.2. Kobe, Japan, 2009, p. 5.
- [10] A. Ortiz, F. Bonnin-Pascual, and E. Garcia-Fidalgo, "Vessel inspection: A micro-aerial vehicle-based approach," *Journal of Intelligent & Robotic Systems*, vol. 76, no. 1, pp. 151–167, 2014.
- [11] F. Bonnin-Pascual, E. Garcia-Fidalgo, and A. Ortiz, "Semi-autonomous visual inspection of vessels assisted by an unmanned micro aerial vehicle," in *IEEE/RSJ IROS*, 2012, pp. 3955–3961.
- [12] A. Ortiz, F. Bonnin-Pascual, E. Garcia-Fidalgo *et al.*, "Vision-based corrosion detection assisted by a micro-aerial vehicle in a vessel inspection application," *Sensors*, vol. 16, no. 12, p. 2118, 2016.
- [13] Mavros. (2019) Mavros. [Online]. Available: <http://wiki.ros.org/mavros>
- [14] G. Bradski and A. Kaehler, *Learning OpenCV: Computer vision with the OpenCV library*. "O'Reilly Media, Inc.", 2008.
- [15] L. Meier, P. Tanskanen, F. Fraundorfer, and M. Pollefeys, "Pixhawk: A system for autonomous flight using onboard computer vision," in *IEEE ICRA*, 2011, pp. 2992–2997.
- [16] ArduPilot. (2019) Ardupilot. [Online]. Available: www.ardupilot.org
- [17] Z. Wrapper. (2019) Zed wrapper. [Online]. Available: github.com/stereolabs/zed-ros-wrapper
- [18] T. Gupta and H. Li, "Indoor mapping for smart cities—an affordable approach: Using kinect sensor and zed stereo camera," in *IPIN*, 2017, pp. 1–8.
- [19] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *IEEE/RSJ IROS*, vol. 3, 2004, pp. 2149–2154.
- [20] F. Furrer, M. Burri, M. Achtelik, and R. Siegwart, "Rotors—a modular gazebo mav simulator framework," in *Robot Operating System (ROS)*, 2016, pp. 595–625.
- [21] R. Smith, "Tesseract ocr engine," *Lecture. Google Code*. Google Inc, 2007.
- [22] Z. Zhang, "Introduction to machine learning: k-nearest neighbors," *Annals of translational medicine*, vol. 4, no. 11, 2016.